GUÍA DE ANÁLISIS Y DISEÑO

Desafío II — Fiesta mundial con UdeAWorldCup

Informática II · Semestre 2026-1

Análisis del problema

Este desafío consiste en modelar mediante la programación orientada a objetos la copa mundial de fútbol 2026, el sistema debe gestionar información de los 48 equipos, los 26 jugadores, los 104 partidos del torneo y como se progresa a través de cada etapa. Con esta simulación lo que se quiere es generar resultados a partir de modelos probabilisticos ya definidos en el enunciado y actualizar las estadisticas correspondientes.

Cuando analizamos el problema se identificaron 5 funcionalidades que dan a la solución, la primera es la carga y actualización de los datos, la segunda la conformación de los grupos, la tercera es una simulación de partidos por etapa con sus reglas, la cuarta la creación de un informe estadístico final y la quinta y última es la medición del consumo de recursos, cada una debe ser accesible de forma separada en un menú.

Hay varios puntos a tomar en cuenta para no sobrecargar el proyecto, sede única con nombre fijo, 3 árbitros con códigos fijos, hora fija en todos los partidos y fecha para todas las etapas después a la fase de grupos, los jugadores se fabrican artificialmente con etiquetas de nombre y apellido dependiendo del número de la camiseta.

Restricciones de la implementación

- Se prohíbe el uso de contenedores de STL y de strucs.
- Prohibido el uso de librerias de terceros que resuelvan cualquier punto esencial del proyecto.
- Memoria dinámica obligatoria.
- Diseño modular multi archivo
- Se permite librería estándar para generar número aleatorios como random o cstdlib.
- No se debe usar almacenamiento permanente.

Conceptos OOP que se deben resaltar

Se exige el uso d abstracción, encapsulación, relaciones entre clases, diagrama de clases UML, funciones amigas, sobrecarga de métodos, una sobre carga de al menos 2 operadores y usoi de plantillas. En el diagrama debe quedar claro la ubicación de cada uno de estos puntos y su respectiva explicación.

Entidades y responsabilidades

- Confederación = Agrupa los equipos por la ubicación geográfica, esto permite aplicar restricciones en los sorteos.
- Equipo = Respresenta una selección nacionál, esta clase contiene a su plantilla, su director técnico, ranking FIFA y estadísticas históricas.
- Jugador = Representa a un jugador de un equipo, guarda los datos personales e historial de estadísticas.
- EstadisticaEquipo = Encapsula los contenedores historicos de un equipo.
- Partido = Modela un encuentro entre dos equipos, tiene fecha, hora, sede, árbitros, convocados por equipo y un resultado simulado.
- EstadisticaPartido = Guarda la información de un equipo en un partido puntual.
- RegistroJugadorPartido = Guarda la información o desempeño de un jugador en un partido.
- Grupo = Contiene 4 equipos en la fase de grupos y calcula la tabla de clasificación al terminar los 6 partidos.
- Bombo = Es un contenedor temporal que tiene 12 equipos, se usa para en sorteo inicial.
- Fecha = Es una clase que permite una aritmética más sencilla, diferencia en días, avanzar Ndías.
- Torneo = Carga los datos, realiza el sorteo, coordina las etapas y produce el informe final, es el organizador de todo.
- MedidorRecursos = Lleva el conteo de las iteraciones y un aproximado de la memoria consumida por funcionalidad
- Lista<T> = Es una platilla de contenedor propio que se bassa en la memória dinámica, sustituye contenedores de la STL.

Nota

La Sede y Árbitro no se modelan como clases propias, ya que su representación es de cadenas fijas, el incluirla como clase sería agregar dificultad sin beneficio alguno.

Uso de plantillas

Se va a implementar una clase plantilla ListaT basada en memoria dinámica, esta plantilla se va a instanciar en varios lugares, Lista<Equipo*> en Torneo y Grupo, Lista<Jugador> en Equipo, Lista<Partido*> en Torneo, Lista<RegistroJugadorPartido> en EstadisticaPartido, entre otras. Usar una plantilla evita reimplementar el contenedor para cada tipo.

Sobrecarga de operadores

Candidatos

- operator<< (función amiga) en Equipo, Partido y Grupo: permite imprimir objetos con sintaxis natural std::cout << equipo.

Cumple simultáneamente el requisito de función amiga.

- operator< en Equipo: habilita ordenar la lista por ranking FIFA para la conformación de bombos.
- operator== en Equipo: permite comparar equipos por identificador (por ejemplo código de país) de forma legible.
- operator+= en EstadisticaEquipo y EstadisticaJugador: acumular sobre el historico los resultados de un nuevo partido con expresión concisa.
- operator[] en Lista<T>: acceso por índice al estilo de un arreglo, manteniendo encapsulación.

Funciones amigas

Por ejemplo, la sobrecarga de OPERATOR se tiene que implementar como función amiga ya que el operador izquierdo que es std::ostream no es una instancia de clase.

Regla de los tres

Cada una de las clases que vaya a manejar memoria din[amica tiene que implementar un constructor de copia, operador de asignación y destructor para evitar superficiales, esto aplica a ListaT, Equipo, Torneo y cualquier clase que tenga punteros a algunos recursos.

Tipos de estructuras de datos

Uso	Estructura propuesta	Justificación
Plantilla de 26 jugadores	Lista<Jugador> (arreglo dinámico)	Tamaño fijo y conocido, se recorre completa para elegir 11 convocados. El acceso aleatorio facilita el muestreo.
48 equipos del torneo	Lista<Equipo*> (arreglo dinámico de punteros)	Tamaño fijo. Se ordena por ranking FIFA para los bombos; el acceso por índice es frecuente.
Equipos de un Grupo (4)	Lista<Equipo*>	Tamaño fijo pequeño. Se recorre para generar la tabla de posiciones.
Partidos de una etapa	Lista<Partido*>	Cantidad conocida por etapa. Se recorre secuencialmente para simular e imprimir.
11 convocados de un partido	Lista<RegistroJugadorPartido>	Tamaño fijo. No hay cambios durante el partido.
Equipos de un Bombo (12)	Lista<Equipo*>	Tamaño fijo. Se eliminan elementos al azar; la reducción del tamaño es manejable con shift o swap-and-pop.
Goleadores por partido	Lista<int> (números de camiseta)	Cantidad pequeña y variable, acceso secuencial al imprimir.

La elección de la estructura de datos se hace caso por caso dependiendo del tamaño que se conozca, el patrón de acceso y frecuencia de inserción/eliminación, el criterio

general es, que, si el tamaño es conocido y fijo, arreglo dinámico redimensionable una sola vez, si hay quizá inserciones, lista enlazada, si se requiere acceso aleatorio y ordenado frecuente, un arreglo dinámico.

¿Por qué un arreglo dinámico y no una lista enlazada para la mayoría de los casos?

En casi todos los usos de nuestro sistema los tamaños son conocidos, entonces bajo esta premisa un arreglo dinámico ofrece acceso aleatorio, que es, menor localidad de caché y menor sobrecarga por elemento que una sola lista enlazada

Paso de referencias y paso por valor, nuestros objetos estructurados, Equipo, Partido, Jugador, ListaT, tienen que pasar por referencia para evitar copias que conlleven a consumos altos. Los tipos primitivos que son, int, bool, double se pasan por valor. Donde una colección debe referenciar entidades compartidas se usan punteros, no copias.

Formato propuesto para el archivo de juagdores

Se exige diseñar un formato propio para para persistir las estadisticas historicas de los jugadores, se propone un archivo de texto con separador ¨;¨ y una línea por jugador con el siguiente orden de campos:

codigoEquipo;numeroCamiseta;nombre;apellido;partidosJugados;goles;minutos;asistencias;amarillas;rojas;faltas

El texto plano nos permite inspección y depuración manual, el separador evita colisiones con posibles comas internas, el prefijo codigoJugador deja reconstruir la relacion con el equipo al leer, se evita JSON o XML para no usar librerias externas.

Conformación de bombos y sorteo de grupos

Entrada: lista de 48 equipos con ranking FIFA y confederación. Salida: 12 grupos de 4 equipos respetando las restricciones.

Paso 1. Colocar al equipo anfitrión (EE. UU.) directamente en el bombo 1.

Paso 2. Ordenar los 47 equipos restantes por ranking FIFA ascendente.

Paso 3. Distribuir en los 4 bombos: los 11 primeros completan el bombo 1, los siguientes 12 al bombo 2, y así sucesivamente.

Paso 4. Por cada uno de los 12 grupos, extraer un equipo al azar de cada bombo y validar restricciones: no repetir confederación salvo UEFA (máximo dos). Si falla, devolver el equipo al bombo e intentar con otro.

Paso 5. Si tras varios intentos no se consigue asignar, aplicar backtracking: deshacer la última asignación y reintentar. Esto previene bloqueos por combinaciones imposibles.

Asignación de fechas en la fase de grupos

Restricciones: 19 días disponibles (20/jun al 8/jul de 2026), máximo 4 partidos por día, ningún equipo puede jugar dos partidos dentro de un lapso de 3 días consecutivos.

Generar los 72 partidos de la fase de grupos (6 por grupo).

Recorrer los partidos en orden e intentar asignarle la primera fecha válida en el rango.

Para cada candidato a fecha: verificar que ese día no tenga ya 4 partidos, y que ninguno de los dos equipos haya jugado en los 3 días previos o los 3 siguientes.

Si no se encuentra fecha, retroceder y replantear (backtracking simple).

Simular un partido en la fase de grupos

- Calcular λ_A y λ_B con la ecuación usando los promedios históricos de goles a favor y en contra.
- Para cada equipo, elegir aleatoriamente 11 jugadores de los 26.
- Para cada jugador, simular amarillas (6% primera, 1.15% segunda, si dos, roja), faltas (13%, 2.75%, 0.7%), y goles (4% por jugador hasta agotar λ).
- Posesión: modelo proporcional al ranking FIFA (por ejemplo, posesión_A = rank_B / (rank_A + rank_B) × 100; ajustar para que sume 100).
- Registrar resultado y actualizar estadísticas históricas de ambos equipos y jugadores convocados.

Simulación de etapas eliminatorias

Igual que en la de grupos, pero en este caso si es empate se desempata por prórroga, se suma 1 al λ del equipo con mejor ranking FIFA, se repite el sorteo y los minutos de todos los jugadores se registran como 120 een lugar de 90.

Clasificación y transición a dieciseisavos

- Calcular puntos por grupos
- Ordenar grupos por puntos, diferencia de goles een el grupo, goles a favor en el mundial, sorteo aleatorio.
- Clasifican los 12 primeros, los 12 segundos y los 8 mejores terceros.
- Emparejar según las reglas, cabezas de grupo vs terceros, el resto de cabezas vs peores segundos, el resto de segundos entre sí, no se pueden enfrentar equipos del mismo grupo entre sí.

Medición de recursos

Se propone una clase MedidorRecursos con dos contadores, iteraciones y memoria estimada, cada clase que asigne o libere memoria notifica al medidor. Los bucles críticos aumentan el contador de iteraciones, al regresar al menú principal después de cada funcionalidad se imprime el resumen y se resetea el contador de iteraciones.

Problemas previstos

- El sorteo de grupos con la restricción UEFA puede llevar a problemas si se asigna con greedy puro, la solución sería un backtracking con límite de intentos.

- Cuando se asignan las fechas en la fase de grupos se pueden generar algunos conflictos al llenar la ventana de 19 días, la solución sería ordenar partidos por grado de restricción antes de asignar.
- La medición de memoria hay que hacerla manual.